

DOCUMENTACAO TECNICA

Otimizador - QuantVision

MVP para comparacao de algoritmos de otimizacao de carteiras com ativos da B3

Campo	Descricao
Repositorio analisado	C:\Users\Henrique Yuji\Desktop\Otimizador
Data do documento	25 de maio de 2026
Status de verificacao	pytest -q executado com sucesso: 14 testes passaram em 7,39s
Entrega	Documento PDF explicando visao, arquitetura, fluxos, APIs, operacao e riscos

Leitura rapida

O QuantVision e uma aplicacao full-stack em Python + frontend web que baixa precos historicos, calcula features financeiras, executa tres algoritmos de otimizacao, compara os resultados e gera saidas em API, CSV/JSON, graficos e PDF.

1. Resumo executivo

O projeto Otimizador, apresentado no frontend como QuantVision, é um MVP de análise e otimização de carteiras financeiras. Ele foi construído para comparar diferentes técnicas de otimização usando os mesmos dados de entrada e a mesma função objetivo, permitindo avaliar pesos sugeridos, retorno esperado, volatilidade, índice de Sharpe, contribuição de risco e tempo de execução.

A solução combina uma biblioteca Python de domínio, uma API local em FastAPI, handlers AWS Lambda, uma interface web estática e scripts de apoio para empacotamento, experimentos, gráficos e relatórios acadêmicos.

Aspecto	Descrição
Produto	MVP de otimização e comparação de carteiras da B3
Entrada principal	Tickers, período ou janela de datas, intervalo, aversão a risco e peso máximo por ativo
Fonte de dados	Yahoo Finance via yfinance, com cache local CSV e cache opcional em S3
Algoritmos	Linear programming aproximado por busca numérica, algoritmo genético e simulated annealing
Saídas	JSON, CSV, dashboard frontend, gráficos e relatório PDF
Ambientes	Execução local, API FastAPI, AWS Lambda + API Gateway, S3 e CloudWatch

2. Como explicar o projeto em 30 segundos

Pitch

O QuantVision recebe uma lista de ativos da B3, coleta dados históricos, calcula retorno, risco e covariância, executa três técnicas de otimização sobre a mesma função objetivo e mostra qual abordagem gerou a melhor carteira segundo retorno ajustado ao risco. O sistema também exporta os resultados e gera relatórios para apresentação técnica ou acadêmica.

- Para usuários: uma interface para simular carteiras e comparar métodos.
- Para avaliadores técnicos: um backend modular com domínio, algoritmos, ingestão, APIs e testes.
- Para apresentação acadêmica: gráficos, LaTeX, PDFs e explicação de arquitetura.

3. Arquitetura geral

A arquitetura separa responsabilidades em camadas. O domínio define modelos e objetivo de otimização; a camada de dados busca e transforma preços; a aplicação orquestra o experimento; a infraestrutura expõe a lógica por API local e handlers Lambda; e a camada de avaliação produz comparações, exports e PDFs.

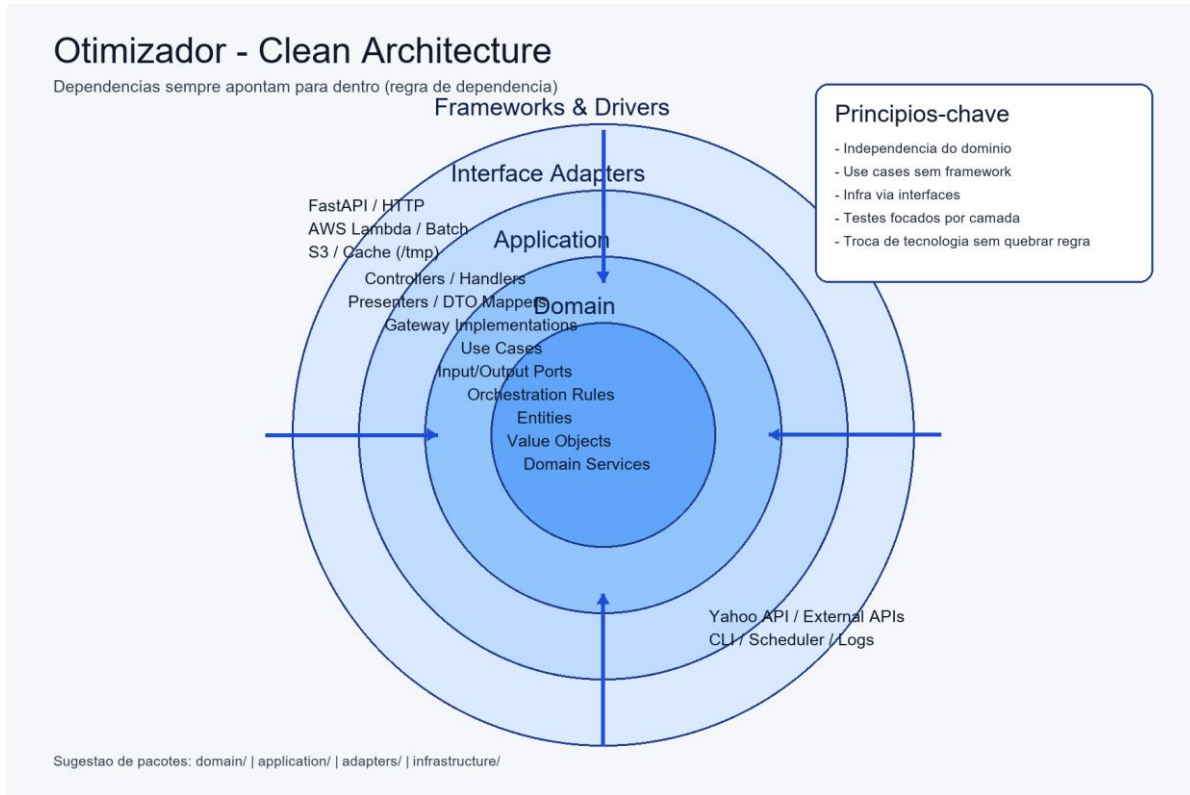


Figura 1 - Diagrama de arquitetura limpa usado como referência visual do projeto.

Camada	Responsabilidade	Arquivos principais
Domain	Modelos, configuração, função objetivo e schema de saída	src/otimizador/domain
Data	Ingestão via yfinance, cache e feature engineering	src/otimizador/data
Algorithms	Implementações LP, GA e SA	src/otimizador/algorithms
Application	Orquestração ponta a ponta do experimento	src/otimizador/application.py
Infrastructure	FastAPI local, handlers Lambda e helpers HTTP	src/otimizador/infrastructure
Evaluation	Comparação, exportação e PDF	src/otimizador/evaluation
Frontend	SPA estática QuantVision	frontend/index.html

4. Fluxo ponta a ponta

1. Usuário seleciona ativos, datas, intervalo e peso máximo no frontend ou envia payload para a API.
2. Backend monta a configuração a partir de variáveis de ambiente e parâmetros da requisição.
3. Ingestão busca preços pelo yfinance; se falhar, tenta cache CSV local; em Lambda pode baixar cache do S3 antes da execução.

4. Feature engineering calcula retornos, retorno esperado, volatilidade e matriz de covariancia.
5. A aplicacao executa linear_programming, genetic_algorithm e simulated_annealing com a mesma funcao objetivo.
6. Cada algoritmo retorna pesos normalizados, valor objetivo, retorno esperado, metadados de risco e tempo de execucao.
7. A avaliacao ordena os resultados por objective_value, escolhe o vencedor e gera ranking e resumo.
8. O resultado e exibido no frontend ou exportado em JSON, CSV e PDF.

5. Dominio e modelo de dados

Os principais contratos de dominio estao em `src/otimizador/domain/models.py`. Eles tornam a saida dos algoritmos comparavel e ajudam a manter a API estavel.

Modelo	Papel
FeatureSet	Agrupa nomes dos ativos/features, retorno esperado, volatilidade, matriz de covariancia e frame de retornos.
OptimizationRequest	Entrada padronizada para os algoritmos, incluindo pesos maximos, seed, aversao a risco e metadados.
AlgorithmResult	Saida padronizada com algoritmo, objetivo, retorno esperado, pesos, tempo e metadados.
ALGORITHM_OUTPUT_SCHEMA	Contrato minimo validado antes de retornar resultados ao usuario.

A funcao objetivo comum e `linear_risk_adjusted_return`: retorno esperado da carteira menos uma penalidade proporcional a volatilidade. Em termos conceituais, o sistema recompensa retorno e penaliza risco conforme o parametro `risk_aversion`.

6. Dados, cache e features

A ingestao esta em `src/otimizador/data/ingestion.py`. O codigo tenta baixar precos de fechamento por `yfinance` usando varias estrategias: `download` em lote com `auto_adjust`, tentativa sem `auto_adjust`, `fallback` de periodo mais robusto e historico individual por `ticker`. Isso reduz falhas de download vazio.

- Cache local: arquivos CSV em `cache_dir`, por padrao `cache/`.
- Nome do cache: simbolos + janela de datas ou periodo + intervalo, por exemplo `PETR4_SA__VALE3_SA__ITUB4_SA_20150101_20251231_1d.csv`.
- Cache em Lambda: o handler de otimizacao pode baixar o CSV de um bucket S3 para `/tmp/cache` antes de executar.
- Fallback sintetico: `OTIMIZADOR_ALLOW_SYNTHETIC_DATA` permite gerar precos deterministico-sinteticos quando habilitado.

Depois da ingestao, `src/otimizador/data/features.py` calcula retornos percentuais diarios, media dos retornos, desvio padrao populacional e matriz de covariancia.

7. Algoritmos de otimizacao

Algoritmo	Como funciona	Metadados relevantes
linear_programming	Busca numerica multi-start com amostras Dirichlet e refinamento local hill climbing. Mantem compatibilidade com Lambda sem scipy.	solver=numpy_multistart_hillclimb
genetic_algorithm	Populacao inicial aleatoria normalizada, selecao por torneio, elite, crossover e mutacao gaussiana.	population_size, generations, seed
simulated_annealing	Perturba pesos normalizados e aceita movimentos conforme energia, temperatura e taxa de resfriamento.	iterations, initial_temperature, cooling_rate, seed

Todos os algoritmos passam pelo mesmo normalizador de pesos. O normalizador impede pesos negativos, forca soma igual a 1 e respeita max_weight. Se max_weight multiplicado pela quantidade de ativos for menor que 1, a configuracao e considerada inviavel.

8. API local e handlers AWS

A API local FastAPI esta em src/otimizador/infrastructure/local_api.py e e pensada para desenvolvimento e uso do frontend local. Os handlers AWS ficam em src/otimizador/infrastructure/handlers e expõem a mesma logica em Lambda.

Endpoint	Metodo	Ambiente	Descricao
/status	GET	FastAPI e Lambda	Healthcheck simples do servico.
/optimize	POST	FastAPI e Lambda	Executa otimizacao; pode retornar todos os algoritmos ou um algoritmo selecionado.
/export	POST	FastAPI	Executa experimento e grava JSON/CSV em diretorio configurado.
/report/pdf	POST	FastAPI e Lambda	Executa experimento, exporta resultados, gera graficos quando solicitado e retorna PDF.
/data	GET	Lambda/SAM	Carrega dados e retorna resumo de linhas/colunas.
/report	GET	Lambda/SAM	Retorna resumo com vencedor, ranking e quantidade de resultados.

9. Frontend QuantVision

O frontend principal e uma pagina HTML estatica autocontida em frontend/index.html. Ele usa uma URL de API configuravel e se comunica com os endpoints do backend por fetch.

- Inicio: mostra status, quantidade de ativos e ultimo resultado.
- Selecao de Ativos: entrada de tickers, periodo, datas e peso maximo.
- Resultados: exhibe metricas, composicao do portfolio, pesos e observacoes tecnicas.
- Comparacao: apresenta ranking, vencedor, alocoes por metodo e recomendacao tecnica.

- Dashboard: resume parametros, fronteira risco-retorno e tabela de performance.
- Exportacao: baixa CSV, JSON e solicita PDF via API.

Observacao sobre o frontend

Existem tambem frontend/app.js e frontend/styles.css, que parecem uma versao alternativa ou anterior da interface. O HTML atual contem CSS e JavaScript inline e parece ser a experiencia mais completa.

10. AWS, cache em S3 e CI/CD

A arquitetura cloud usa API Gateway, Lambda, S3 para artefatos/cache e CloudWatch Logs para observabilidade. O template SAM em docs/infrastructure.template.yaml define uma HttpApi e funcoes para data, optimize, report e status. O workflow GitHub Actions .github/workflows/ci-cd-lambda.yml executa lint/testes e, em push na main, empacota e atualiza Lambdas em ambiente dev.

Componente	Papel
API Gateway	Entrada HTTP publica para os endpoints.
AWS Lambda	Executa handlers de dados, otimizacao, relatorio, PDF e status.
S3	Armazena pacotes de deploy e pode armazenar cache CSV de precos.
CloudWatch Logs	Diagnostico de falhas de ingestao, empacotamento e execucao.
GitHub Actions	Roda ruff, pytest, build em imagem Lambda Python 3.11, upload S3 e update-function-code.

11. Relatorios, graficos e artefatos academicos

A pasta docs contem materiais academicos e de apresentacao, incluindo LaTeX, PDFs, diagramas UML e graficos gerados. O script scripts/generate_tcc_charts.py produz dez figuras principais para explicar dados, risco, pesos, metricas, fronteira risco-retorno, convergencia, backtest e drawdown.

Arquivo ou pasta	Uso
docs/figures	Graficos e diagramas usados em TCC/apresentacao.
docs/reports	PDFs de relatorio ja gerados e o PDF deste documento.
examples/exports	JSON e CSV de execucoes anteriores.
docs/tcc_otimizacao_abnt.tex	Documento academico em LaTeX.
scripts/generate_pdf_report.py	Gera PDF a partir dos exports e graficos.

12. Como executar localmente

Instalacao basica no Windows/PowerShell:

```
python -m venv .venv
.\venv\Scripts\activate
pip install -r requirements.txt -r requirements-dev.txt
```

Subir a API local:

```
$env:PYTHONPATH='src'  
python -m uvicorn otimizador.infrastructure.local_api:app --host 0.0.0.0 --port 8000 --  
reload
```

Subir o frontend estatico:

```
cd frontend  
python -m http.server 8080
```

Executar experimento local e exportar arquivos:

```
$env:PYTHONPATH='src'  
python scripts/run_local_experiment.py --symbols PETR4.SA,VALE3.SA,ITUB4.SA --start-date  
2015-01-01 --end-date 2025-12-31 --interval 1d --max-weight 0.6
```

13. Testes e qualidade

A suite de testes foi executada durante a preparacao deste documento com o comando `pytest -q`.
Resultado observado: 14 testes passaram em 7,39 segundos.

Area testada	Exemplos de validacao
Algoritmos	Pesos somam 1, respeitam max_weight e retornam valores numericos.
Features	Gera nomes, retorno esperado e volatilidade para os ativos.
Ingestao	Baixa dados, grava cache e usa cache quando rede falha.
Comparacao	Ordena por objective_value e define winner/ranking.
Exportacao	Cria JSON completo, CSV de resumo e CSV de pesos.
Handlers	Status, optimize, data, report e PDF retornam estruturas esperadas.
API local	Endpoint /report/pdf retorna application/pdf.

```
$env:PYTHONPATH='src'  
pytest -q
```

14. Configuracao por variaveis de ambiente

Variavel	Finalidade
OTIMIZADOR_SYMBOL / OTIMIZADOR_SYMBOLS	Ativo unico ou lista de ativos separados por virgula.
OTIMIZADOR_PERIOD	Periodo usado quando nao ha start_date/end_date.
OTIMIZADOR_START_DATE /	Janela historica explicita.

OTIMIZADOR_END_DATE	
OTIMIZADOR_INTERVAL	Intervalo dos dados, por exemplo 1d.
OTIMIZADOR_CACHE_DIR	Diretorio de cache local; em Lambda normalmente /tmp/cache.
OTIMIZADOR_RISK_AVERSION	Peso da penalidade de risco na funcao objetivo.
OTIMIZADOR_MAX_WEIGHT	Peso maximo permitido por ativo.
OTIMIZADOR_RANDOM_SEED	Seed para reproducibilidade.
OTIMIZADOR_GA_POPULATION / OTIMIZADOR_GA_GENERATIONS	Parametros do algoritmo genetico.
OTIMIZADOR_SA_ITERATIONS / TEMPERATURE / COOLING_RATE	Parametros do simulated annealing.
OTIMIZADOR_S3_CACHE_BUCKET / PREFIX	Cache CSV opcional no S3 para ambiente Lambda.
OTIMIZADOR_ALLOW_SYNTHETIC_DATA	Habilita geracao sintetica em caso de falha de ingestao.

15. Pontos de atencao e riscos conhecidos

- O README.md esta com marcadores de conflito de merge, entao precisa ser limpo antes de apresentacao ou entrega formal.
- O docs/docker-compose.yml referencia frontend/Dockerfile, mas esse arquivo nao aparece na listagem atual do projeto.
- Ha duas possiveis linhas de frontend: index.html autocontido e app.js/styles.css separados. Vale escolher uma como oficial.
- A disponibilidade do Yahoo Finance pode variar; por isso o cache CSV e o cache S3 sao importantes para reproducibilidade.
- O template SAM atual lista data, optimize, report e status; o handler de PDF existe e e usado no CI/CD, entao convem alinhar template, README e deploy real.
- O algoritmo chamado linear_programming nao usa solver linear classico; ele usa busca numerica para manter o pacote leve em Lambda. Isso deve ser explicado em apresentacoes tecnicas.

16. Proximos passos recomendados

9. Resolver o conflito do README e transformar este PDF em uma versao resumida de README.md + docs/arquitetura.md.
10. Definir oficialmente qual frontend sera mantido: SPA autocontida ou arquivos separados.
11. Adicionar ou ajustar Dockerfile caso o fluxo Docker seja necessario.
12. Alinhar docs/infrastructure.template.yaml com todos os handlers realmente publicados, incluindo report-pdf se for parte da entrega.
13. Adicionar exemplos de payload e respostas no README para /optimize, /export e /report/pdf.
14. Considerar persistencia de resultados em S3/DynamoDB e historico de execucoes para auditoria.

17. Conclusao

O QuantVision/Otimizador esta em bom estado para demonstracao tecnica: a logica principal esta modularizada, os testes passam, ha frontend, API, handlers Lambda, exportacao e relatorio. O maior trabalho pendente nao e na engine principal, mas na consolidacao da documentacao, limpeza de conflitos e alinhamento entre artefatos de deploy, frontend e README.

Resumo final

Este projeto mostra uma cadeia completa de produto de dados: coleta, cache, transformacao, otimizacao, comparacao, visualizacao, exportacao, PDF e deploy cloud. Para uma banca ou revisao tecnica, essa narrativa e o ponto mais forte da entrega.